MALLA REDDY VISHA VIDYAPEETH

ALGOFRGE HACKATHON

B.Tech II Year • DAA External Hackathon

STUDENT RESULT ANALYSER

Simple, Fast and Explainable Student Performance Analysis

Project	Details
Problem Statement	STUDENT RESULT ANALYSER
Algorithms	Sorting • Binary Search • Graphs
Technologies	Python • Flask • SQLite • Chart.js
Team	Team 6
Academic Year	2026–2027

Team Members

Role	Name	Roll Number
Team Leader	Nandyala Sravan Kumar	25MVCSMR0692
Member	Neela Kamal Raj	25MVCSMR0694
Member	Lambola Yeshwanth Yadav	25MVCSMR0672
Member	Gera Akhil Chaitanya	25MVCSMR0645

“Turning student marks into useful information using algorithms.”

ABSTRACT & INTRODUCTION

1. Abstract

Student Result Analyser is a web-based project that helps analyze student marks in a simple and organized way. The system takes student details and subject marks as input and produces totals, averages, rankings, search results, range results and subject relationship graphs.

The main purpose is to demonstrate how Design and Analysis of Algorithms (DAA) can solve a real problem. Sorting is used to arrange students and create rankings. Binary Search is used to find a student quickly from sorted data. Graphs are used to show relationships between subjects.

Python and Flask handle the backend, SQLite stores the records, and Chart.js displays results visually. This makes the project both algorithm-focused and practical.

2. Why This Project Is Needed

When there are many students, manually calculating totals, rankings and searches takes time. A result analyser automates these tasks and gives the user a clear view of performance.

3. Project Vision

- Store marks in an organized format.
- Calculate total and average automatically.
- Rank students using sorting.
- Find students using Binary Search.
- Answer marks-based range queries.
- Show subject relationships using graphs.
- Present results through tables and charts.

4. Simple System Flow

```
Student Data → SQLite → Algorithms → Analysis → Charts / Results
```

PROBLEM STATEMENT, OBJECTIVES & SCOPE

5. Problem Statement

Colleges maintain marks for many students and subjects. Users may want to know the highest scorer, a student's rank, which students fall within a marks range, or which subjects have similar performance patterns. Doing these operations manually becomes difficult as the data grows.

The Student Result Analyser solves this by accepting student marks, storing them, processing them with suitable algorithms and showing the results in an understandable form.

6. Objectives

- Calculate total and average marks correctly.
- Sort students and generate ranking.
- Use Binary Search for efficient lookup.
- Answer range queries.
- Represent subject relationships with a graph.
- Display results using charts.
- Show useful algorithm steps or comparisons.

7. Scope

The project covers data entry, result calculation, ranking, searching, range queries, graph creation and visualization. It is a hackathon prototype, not a complete college examination system.

8. Proposed Solution

The solution has five stages: input, storage, processing, analysis and output. Student data is entered, saved in SQLite, processed by algorithms, converted into useful results and finally displayed through tables and charts.

Stage	What Happens
1. Input	Enter IDs, names and subject marks.
2. Storage	Save records in SQLite.
3. Processing	Calculate totals and run algorithms.
4. Analysis	Generate ranks, searches, ranges and graphs.
5. Output	Show results using tables and Chart.js.

FEATURES & FUNCTIONAL REQUIREMENTS

9. Main Features

Feature	Simple Explanation
Student Data	Stores ID, name and subject marks.
Total & Average	Calculates total and average.
Ranking	Sorts students and assigns ranks.
Binary Search	Quickly searches sorted data.
Range Query	Finds students between selected marks.
Subject Graph	Shows relationships between subjects.
Charts	Displays marks, averages and rankings.
Algorithm Metrics	Shows steps, comparisons or time when required.

10. Functional Requirements

ID	Requirement	Expected Result
FR-01	Enter data	Valid data is accepted.
FR-02	Store data	Records are saved in SQLite.
FR-03	Calculate result	Total and average are generated.
FR-04	Rank	Students are ordered by performance.
FR-05	Search	Binary Search finds a matching record.
FR-06	Range	Matching students are returned.
FR-07	Graph	Subjects and relationships are represented.
FR-08	Visualize	Results appear in tables/charts.
FR-09	Validate	Invalid input is handled safely.

11. Non-Functional Requirements

- Easy to use.
- Fast enough for growing datasets.
- Reliable for the same valid input.
- Modular so algorithms can be changed easily.
- Scalable for more students and subjects.
- Safe input and database handling.

12. User Workflow

Enter data → Calculate results → View ranking → Search a student → Select a marks range → View subject relationships → Check charts and algorithm information.

ALGORITHMS & METHODOLOGY

13. Sorting — Merge Sort

Sorting arranges students according to total marks. Merge Sort uses divide-and-conquer: it divides the list, sorts the smaller parts and merges them.

```
MERGE_SORT(A):
  if length(A) <= 1: return A
  divide A into Left and Right
  Left = MERGE_SORT(Left)
  Right = MERGE_SORT(Right)
  return MERGE(Left, Right)
```

Time complexity: O(n log n) in best, average and worst cases. After sorting, the system can assign ranks from the sorted order.

14. Binary Search

Binary Search works on sorted data. It checks the middle item. If the target is smaller or larger, half of the remaining data is ignored. This continues until the target is found or the search ends.

```
BINARY_SEARCH(A, target):
  low = 0; high = length(A)-1
  while low <= high:
    mid = (low + high) // 2
    if A[mid] == target: return mid
    if A[mid] < target: low = mid + 1
    else: high = mid - 1
  return -1
```

Best case: O(1). Average and worst case: O(log n). The project can count the comparisons made during the search.

15. Range Query

A range query finds students whose marks are between two values. Example: find students with total marks from 350 to 450.

```
Find first record >= lower mark
Find last record <= upper mark
Return records between those positions
```

16. Graphs

In the graph, subjects are vertices and relationships are edges. BFS or DFS can be used to explore the graph.

```
Mathematics ─── Programming
     │                 │
Statistics ──────── DAA
```

For V vertices and E edges, BFS/DFS takes O(V + E) time using an adjacency list.

17. Complete Method

1. Enter and validate marks.
2. Store data in SQLite.
3. Calculate total and average.
4. Sort and create ranking.
5. Run Binary Search.
6. Process range query.
7. Build/traverse subject graph.
8. Display results with Chart.js.

TECHNOLOGIES, IMPLEMENTATION, TESTING & PERFORMANCE

18. Technologies

Technology	Purpose
Python	Algorithms and backend logic.
Flask	Web backend and API layer.
SQLite	Student/result database.
HTML/CSS/JavaScript	User interface.
Chart.js	Interactive charts.

19. System Architecture

```
User Interface
      ↓
Flask Backend / API
      ↓
SQLite Database
      ↓
Algorithm Layer
   ├─ Sorting
   ├─ Binary Search
   ├─ Range Query
   └─ Graph Processing
      ↓
Results + Charts + Metrics
```

20. Database Design

Field	Meaning
student_id	Unique student ID
student_name	Student name
maths / programming / DAA / database	Subject marks
total	Sum of marks
average	Average marks
rank	Ranking position

21. Testing

Test	Situation	Expected Result
TC01	Valid student data	Data stored.
TC02	Subject marks	Total/average correct.
TC03	Ranking	Students sorted and ranked.
TC04	Existing search	Student found.
TC05	Missing search	Not Found shown.
TC06	Marks range	Matching students returned.
TC07	Graph data	Graph created.
TC08	Invalid input	Validation message shown.

22. Performance

Operation	Algorithm	Complexity
Ranking	Merge Sort	O(n log n)
Search	Binary Search	O(log n)
Graph traversal	BFS / DFS	O(V + E)
Simple search	Linear Search	O(n)

The important idea is algorithm choice. Sorting organizes data, Binary Search reduces the search area quickly, and graph traversal efficiently explores subject relationships.

CONCLUSION, FUTURE SCOPE & REFERENCES

23. Conclusion

The Student Result Analyser connects DAA concepts with a real academic problem. It takes student marks and converts them into totals, averages, rankings, search results, range results and subject relationships.

Each main algorithm has a clear purpose. Sorting creates the ranking order. Binary Search provides faster lookup after the data is sorted. Graph algorithms represent and explore relationships between subjects. This makes the project easy to explain during a hackathon evaluation.

Python, Flask, SQLite and Chart.js combine algorithmic processing with full-stack development. The result is a practical prototype that demonstrates both programming and DAA knowledge.

24. Future Scope

- Student/faculty/admin login.
- Semester-wise performance analysis.
- Attendance and internal assessment analytics.
- Advanced subject-correlation graphs.
- Machine-learning performance prediction.
- PDF and Excel reports.
- Cloud deployment.
- Mobile-friendly dashboard.

25. References

9. Cormen et al., Introduction to Algorithms, MIT Press.
10. Horowitz, Sahni and Rajasekaran, Computer Algorithms.
11. Python Documentation.
12. Flask Documentation.
13. SQLite Documentation.
14. Chart.js Documentation.

26. Team Acknowledgement

Team 6 developed this prototype to show how algorithmic problem solving can be applied to student-result analysis. The project combines DAA, backend development, database management and visualization.

THANK YOU

STUDENT RESULT ANALYSER • TEAM 6

Nandyala Sravan Kumar • Neela Kamal Raj • Lambola Yeshwanth Yadav • Gera Akhil Chaitanya